

Partial Derivatives and Gradients

Contents

1. Overview	2
2. Functions of several variables	3
3. Partial derivatives	4
3.1 The definition	4
3.2 Computing partial derivatives	6
3.3 The picture: slopes of two slices	6
4. The gradient	7
4.1 Collecting the partial derivatives	7
4.2 The gradient as an arrow on the contour map	8
4.3 The slope in any direction	10
5. Rules for gradients	11
6. The chain rule with several variables	13
6.1 One outer input, two paths	13
6.2 Two outer inputs: a matrix of partial derivatives	14
7. Checking a gradient numerically	15
8. Summary	18
9. Sources	18
10. Key terms	18

Where this fits: Pipeline map step 0 (Foundations). See the [Course map](#).



- **Builds on:** Derivatives of one variable ([Note MA-061](#)).
- **Leads to:** Jacobian and matrix gradients ([Note MA-063](#)); Hessian and multivariate Taylor ([Note MA-064](#)); Lagrange multipliers, KKT and duality ([Note MA-066](#)); Maximum likelihood estimation (MLE) ([Note MA-070](#)); Backpropagation ([Note DL-015](#)); Gradient descent ([Note DL-017](#)).

1. Overview

Key point: For a function of many inputs, we take one derivative per input with the others held fixed; collected in a row, these partial derivatives form the gradient, an arrow that points straight uphill.

This Note follows Chapter 5 (Section 5.2) of *Mathematics for Machine Learning* (Deisenroth, Faisal and Ong, 2020).

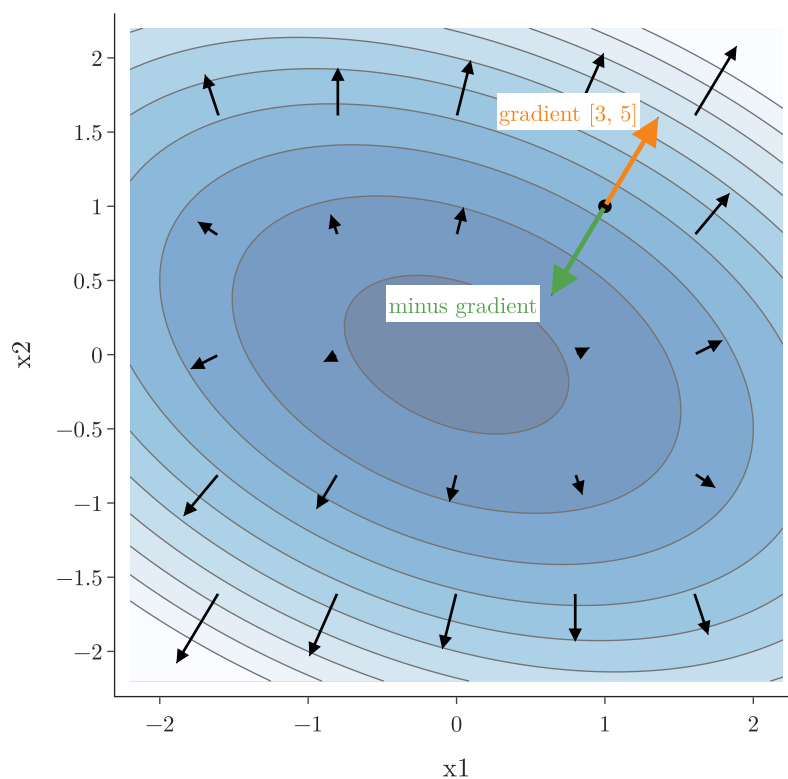


Figure 1: Contour map of $f(x_1, x_2) = x_1^2 + x_1x_2 + 2x_2^2$ with gradient arrows: each crosses the contour lines at right angles and points uphill

A loss depends on many parameters at once. Figure 1 shows such a function of two inputs as a contour map: each line joins points of equal height, and the lowest point is in the middle. At every point, the **gradient** (G-863) is an arrow pointing in the steepest uphill direction. Its opposite, the green arrow, is the step **gradient descent** (G-862) takes.

Earlier Notes already used these ideas:

- the partial derivative as a slope in one variable, from the [linear regression maths Note](#);
- the gradient as the vector of partial derivatives of the loss, from the [gradient descent Note](#).

This Note adds the definition as a limit, the geometry of the gradient, the rules for vectors and the chain rule with several variables. The Note builds on the one-variable derivative of the [derivatives of one variable Note](#).

2. Functions of several variables

Key point: $f : \mathbb{R}^n \rightarrow \mathbb{R}$ takes a vector of n numbers in and gives one number out, just as a loss takes all the parameters and gives one error.

A **function of several variables** (G-815) takes a vector $\mathbf{x} = [x_1, \dots, x_n]^\top$ and returns one number:

$$f : \mathbb{R}^n \rightarrow \mathbb{R}, \quad \mathbf{x} \mapsto f(\mathbf{x})$$

Two examples we use throughout:

- the squared length of a vector, $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{x} = x_1^2 + x_2^2$, the dot product of $\mathbf{x}$ with itself (see the [dot product and cosine similarity Note](#));
- our running example, a tilted bowl like a loss surface:

$$f(x_1, x_2) = x_1^2 + x_1 x_2 + 2x_2^2$$

At $(1, 1)$ its value is $1 + 1 + 2 = 4$.

Figure 2 marks four inputs of the running example on its contour map. Each input is a vector of two numbers, and each gets back a single number: $[1, 1] \mapsto 4$, $[0, 1] \mapsto 2$, $[-1, 0] \mapsto 1$. Two different inputs can give the same number: $[2, -1]$ also gives 4, so it lies on the same contour line as $[1, 1]$.

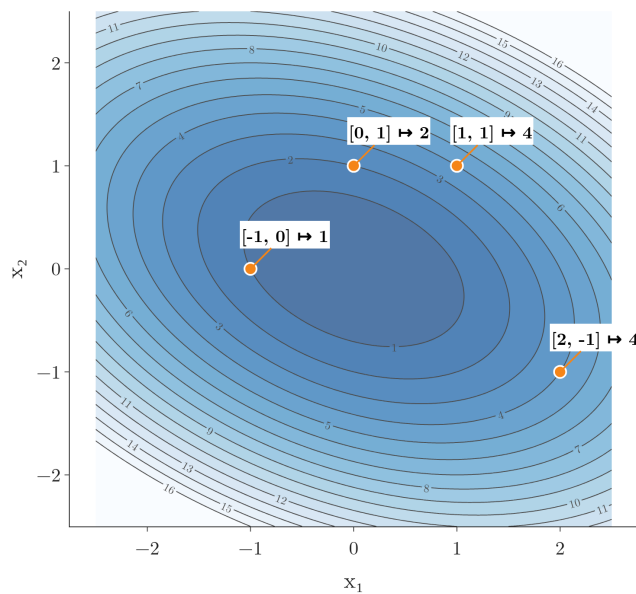


Figure 2: The running example on a contour map, with four input vectors and the single number the function returns for each. $[1, 1]$ and $[2, -1]$ both give 4 and lie on the same contour line.

3. Partial derivatives

Key point: A partial derivative changes one input by a tiny step, keeps all the others fixed, and measures the slope; it is an ordinary one-variable derivative.

3.1 The definition

Key point: Nudge only x_i by h , take the difference quotient, and let $h \rightarrow 0$.

A one-variable derivative answers: if we nudge the input a little, how many times bigger is the change in the output? With two inputs the question needs one more detail: nudge **which** input? Figure 3 tries both on our bowl at $(1, 1)$, where $f = 4$.

1. Nudge only x_1 by 0.1. The output moves from 4 to $f(1.1, 1) = 4.31$: a change of 0.31, about 3 times the nudge.
2. Go back, and nudge only x_2 by 0.1. The output moves from 4 to $f(1, 1.1) = 4.52$: a change of 0.52, about 5 times the nudge.

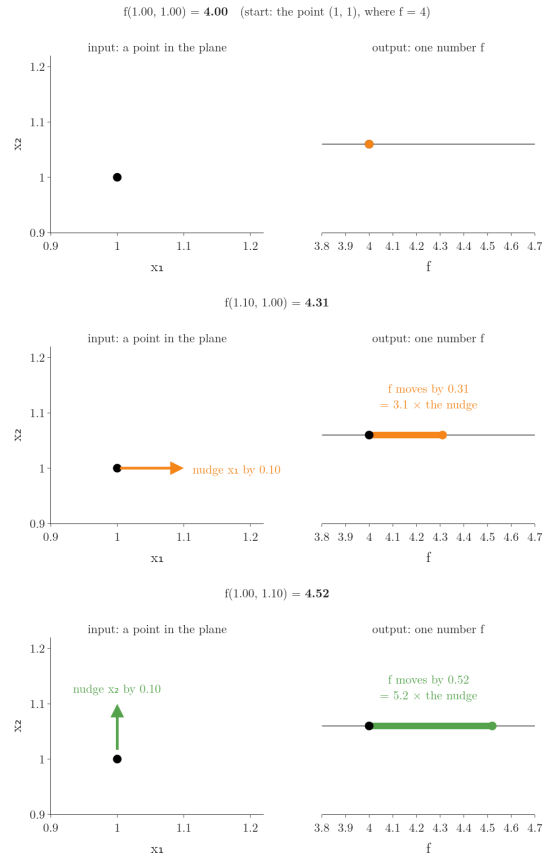


Figure 3: The point $(1, 1)$ in the input plane (left) and the single output number f on a number line (right). A nudge of 0.1 in x_1 alone moves the output by 0.31; the same nudge in x_2 alone moves it by 0.52. Idea after Khan Academy, “Partial derivatives, introduction”

The same function responds differently to each input. Each of the two ratios tells only part of the story of how f changes, which is where the name comes from. The [linear regression maths Note](#) defined the **partial derivative** (G-1457) as the slope of a function of several variables in one variable, holding the others fixed. Written with the limit of the [derivatives of one variable Note](#):

1. **In words:** move only x_i by a step h , divide the change in f by h , and let h shrink to 0.
2. **Formula:**

$$\frac{\partial f}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_i + h, \dots, x_n) - f(\mathbf{x})}{h}$$

The curly ∂ (read “partial”) replaces d to say that the other variables are held fixed.

3. **Example:** for our bowl at $(1, 1)$ with $h = 0.1$,

$$\frac{f(1.1, 1) - f(1, 1)}{0.1} = \frac{1.21 + 1.1 + 2 - 4}{0.1} = 3.1$$

Smaller h brings this towards 3.

3.2 Computing partial derivatives

Key point: Treat every other variable as a constant and use the one-variable rules.

Because the other variables are frozen, they behave like numbers. All the rules of the [derivatives of one variable](#) [Note](#) apply unchanged.

1. **In words:** to differentiate with respect to x_1 , read x_2 as a fixed number, and the other way round.
2. **Formula:** for $f(x_1, x_2) = x_1^2 + x_1x_2 + 2x_2^2$,

$$\frac{\partial f}{\partial x_1} = 2x_1 + x_2, \quad \frac{\partial f}{\partial x_2} = x_1 + 4x_2$$

In $\partial f/\partial x_1$, the term x_1x_2 is x_1 times a constant, so its derivative is that constant x_2 ; the term $2x_2^2$ is a constant, so its derivative is 0.

3. **Example:** at $(1, 1)$: $\partial f/\partial x_1 = 2 + 1 = 3$ and $\partial f/\partial x_2 = 1 + 4 = 5$.

The chain rule works the same way inside a partial derivative. For $g(x, y) = (x + y^2)^2$ at $(1, 2)$, where the inside is $x + y^2 = 5$:

$$\frac{\partial g}{\partial x} = 2(x + y^2) \cdot 1 = 10, \quad \frac{\partial g}{\partial y} = 2(x + y^2) \cdot 2y = 40$$

3.3 The picture: slopes of two slices

Key point: Fixing x_2 cuts the surface along a curve in the x_1 direction; $\partial f/\partial x_1$ is the slope of its tangent line.

Figure 4 shows the surface of our bowl. Fixing $x_2 = 1$ cuts it along the orange curve, a function of x_1 alone. Its tangent line at $(1, 1)$ has slope 3: that is $\partial f/\partial x_1$. Fixing $x_1 = 1$ instead gives the green curve, whose tangent line has slope 5: that is $\partial f/\partial x_2$.

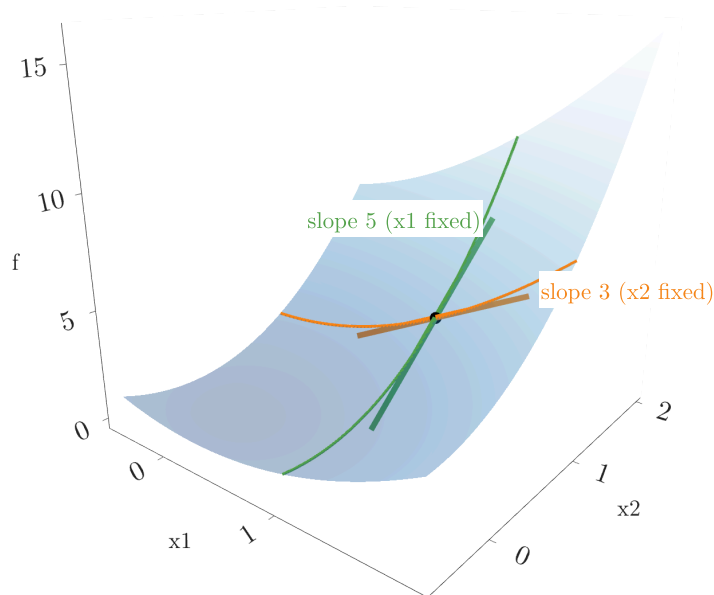


Figure 4: The surface $f(x_1, x_2) = x_1^2 + x_1x_2 + 2x_2^2$ cut along $x_2 = 1$ (orange) and along $x_1 = 1$ (green); the thick lines are the tangent lines at $(1, 1)$

The slice picture needs a surface, so it only works for two inputs. The nudge picture of Figure 3 needs no surface: nudge one input, watch the one output number. That reading still works for a loss with a million parameters.

The two tangent lines together span a flat plane that touches the surface at $(1, 1)$, the tangent plane. The [Hessian and multivariate Taylor Note](#) uses it to approximate the surface.

4. The gradient

Key point: The gradient collects all partial derivatives in one row vector; as an arrow, it points in the direction of steepest ascent and is perpendicular to the contour lines.

4.1 Collecting the partial derivatives

Key point: One partial derivative per input, side by side: a $1 \times n$ row vector.

An earlier Note on descent ([gradient descent Note](#)) defined the **gradient** as the vector of partial derivatives of the loss, pointing in the direction of steepest increase. We write it as a row:

1. **In words:** list the partial derivatives in the order of the inputs.

2. **Formula:**

$$\nabla_{\mathbf{x}} f = \frac{df}{d\mathbf{x}} = \left[\frac{\partial f}{\partial x_1} \quad \frac{\partial f}{\partial x_2} \quad \cdots \quad \frac{\partial f}{\partial x_n} \right] \in \mathbb{R}^{1 \times n}$$

∇ is read “nabla” (G-1295) or “grad”. Written as a row, this is the **gradient as a row vector** (G-858).

3. **Example:** for our bowl,

$$\nabla f = \begin{bmatrix} 2x_1 + x_2 & x_1 + 4x_2 \end{bmatrix}, \quad \nabla f(1, 1) = \begin{bmatrix} 3 & 5 \end{bmatrix}, \quad \nabla f(2, -1) = \begin{bmatrix} 3 & -2 \end{bmatrix}$$

Row or column? Many texts, and the update rules of the [gradient descent Note](#), treat the gradient as a column vector like the parameters. The book we follow uses a row: then the chain rule becomes ordinary matrix multiplication with no transposes (Section 6), and the next Note’s Jacobian follows naturally. The numbers are the same either way; only the shape differs. In NumPy both are plain 1-D arrays, so code rarely notices.

4.2 The gradient as an arrow on the contour map

Key point: Drawn at its point, the gradient arrow points the way f rises fastest, crosses the contour line at a right angle, and is longer where the surface is steeper.

Figure 1 draws the gradient as an arrow at many points of the contour map. Three things are visible:

- **It points uphill.** Every arrow points away from the minimum, towards higher contour lines.
- **It crosses contour lines at right angles.** Along a contour line f does not change at all, so the steepest direction is straight across it.
- **Its length is the steepness.** Arrows are long where contour lines are packed close together, and short near the flat bottom.

The hiker. Think of the surface as a hillside and stand on it. Which way should we walk to climb fastest? Figure 5 answers by doing it. The contour map lies under the bowl, and the orange gradient arrow is drawn on the map under the hiker. At every step the hiker reads the arrow and walks that way. The path heads straight away from the lowest point and gets steeper: the steepness readout, the length of the gradient, grows from 0.9 to 6.5.

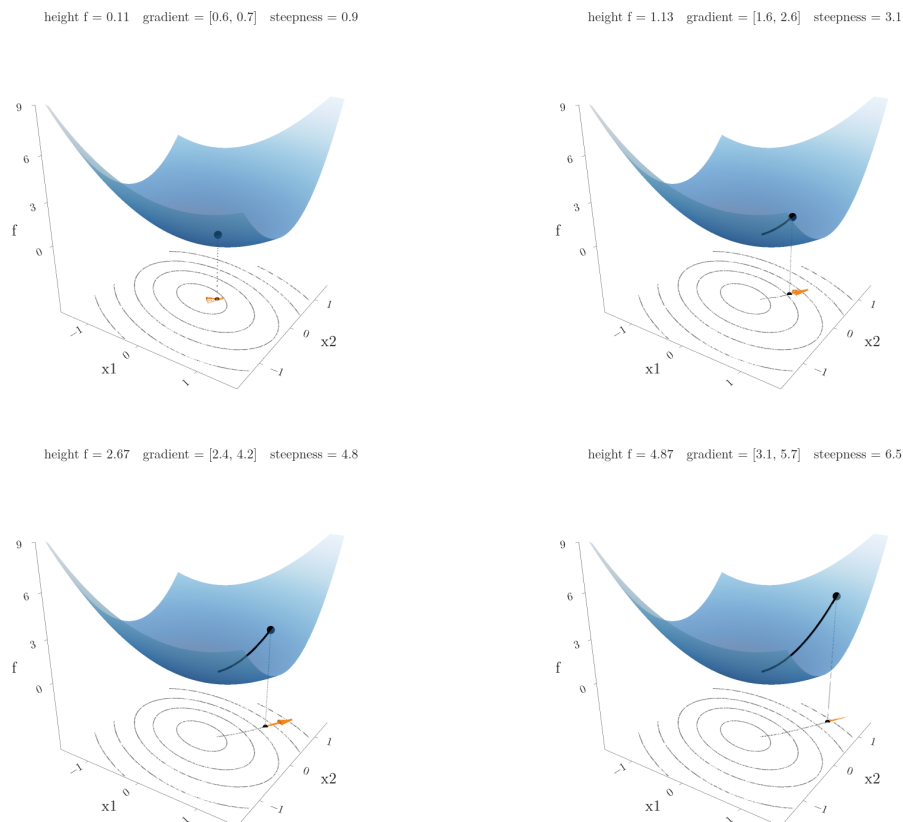


Figure 5: A hiker (black dot) climbs the bowl by always stepping along the gradient arrow (orange) drawn on the contour map below. The dashed shadow of the path crosses every contour line at a right angle, and the arrow grows as the surface gets steeper. Idea after Khan Academy, “Gradient and graphs”; our own function

Why at right angles? Figure 6 zooms in on $(1, 1)$ until the contour lines $f = 4$ and $f = 4.1$ look like two straight, parallel lines.

1. Every arrow from the first line to the second raises f by the same amount, 0.1.
2. The arrows have different lengths: 0.024, 0.019, 0.017, 0.019, 0.024.
3. The steepest direction gains that 0.1 over the shortest distance.
4. The shortest path between two parallel lines goes straight across, at a right angle to both.

So the steepest direction is perpendicular to the contour line, and it is the direction of the gradient $[3, 5]$. The shortest length also checks the third bullet: 0.1 of height over a distance of 0.017 is a slope of $0.1/0.017 \approx 5.83$, the length of the gradient, $\sqrt{3^2 + 5^2}$.

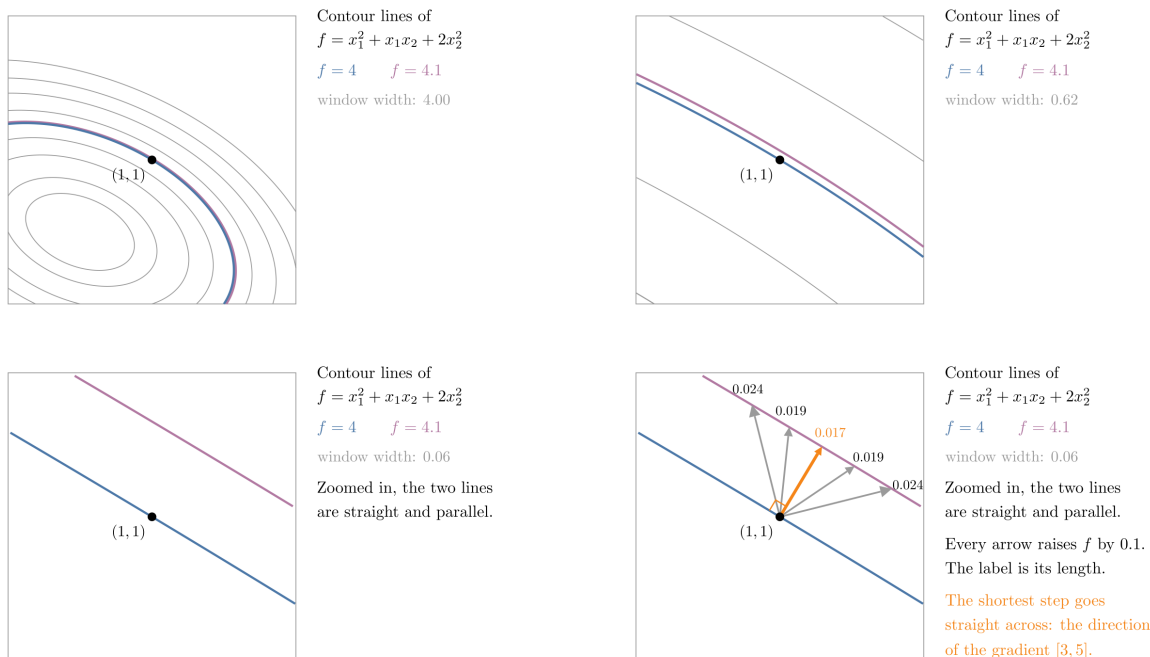


Figure 6: Zooming in on $(1, 1)$: the contour lines $f = 4$ (blue) and $f = 4.1$ (purple) become straight and parallel. Of the arrows that raise f by 0.1, the shortest (orange, length 0.017) crosses at a right angle and points along the gradient [3, 5]. Idea after Khan Academy, “Gradient and contour maps”; our own function

At $(1, 1)$ the gradient $[3, 5]$ (orange) points up and to the right. Its opposite, $[-3, -5]$ (green), points into the bowl: the direction **gradient descent** steps in, $\mathbf{x}_{\text{new}} = \mathbf{x} - \eta \nabla f(\mathbf{x})$.

4.3 The slope in any direction

Key point: The slope of f along a unit vector $\mathbf{u}$ is the dot product $\nabla f \cdot \mathbf{u}$; it is largest when $\mathbf{u}$ points along the gradient.

The partial derivatives give the slope along the two axes. A hiker can walk in any direction, so we need the slope along any arrow.

1. **In words:** a small step along $\mathbf{u} = [a, b]$ is a small steps in x_1 plus b small steps in x_2 . Each x_1 step changes f at the rate $\partial f / \partial x_1$, each x_2 step at the rate $\partial f / \partial x_2$. Add the two.
2. **Formula:** for a unit vector $\mathbf{u} = [a, b]$ (length 1), the **directional derivative** (G-614) is

$$\nabla_{\mathbf{u}} f = a \frac{\partial f}{\partial x_1} + b \frac{\partial f}{\partial x_2} = \nabla f \cdot \mathbf{u}$$

3. **Example:** at $(1, 1)$, where $\nabla f = [3, 5]$:
 - along $\mathbf{u} = [1, 0]$ it is 3, the partial derivative $\partial f / \partial x_1$;
 - along the gradient itself, $\mathbf{u} = [3, 5] / \sqrt{34}$, it is $\sqrt{34} = 5.83$, the largest possible;

- along $\mathbf{u} = [5, -3]/\sqrt{34}$, at right angles to the gradient, it is $(15 - 15)/\sqrt{34} = 0$: we walk along the contour line.

A dot product with a unit vector is the length of the gradient times the cosine of the angle between the two, so it is largest when the two point the same way (see the [dot product and cosine similarity Note](#)). This alignment is why the gradient is the direction of **steepest ascent** (G-1888), and why its length, 5.83, is the slope in that direction: the same number Figure 6 found from the shortest arrow.

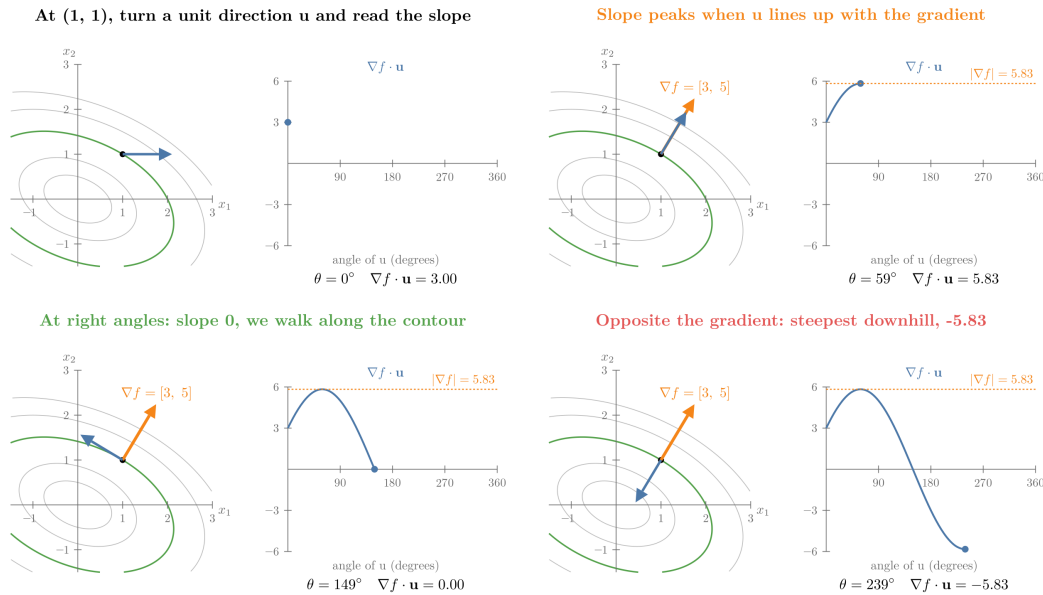


Figure 7: At (1, 1) a unit direction $\mathbf{u}$ turns full circle (left); the slope $\nabla f \cdot \mathbf{u}$ is traced against its angle (right): 3 along the x_1 -axis, a peak of 5.83 when $\mathbf{u}$ lines up with $\nabla f = [3, 5]$, 0 along the contour line, -5.83 straight downhill

Figure 7 turns the three directions above into one sweep. Watch the right panel: the slope rises and falls like a cosine wave, $\sqrt{34} \cos \theta$, where θ is the angle between $\mathbf{u}$ and the gradient. Its top sits exactly where the blue arrow covers the orange gradient arrow, and it crosses 0 where the blue arrow runs along the green contour line. The picture follows Sanderson's Khan Academy lesson on why the gradient is the direction of steepest ascent, redrawn with our function.

5. Rules for gradients

Key point: Sum, product and chain rules still hold with vectors, but the factors are now vectors and matrices, so their order matters.

The rules of the one-variable case carry over. For functions $f, g : \mathbb{R}^n \rightarrow \mathbb{R}$:

$$\text{Sum rule: } \frac{\partial}{\partial \mathbf{x}} (f(\mathbf{x}) + g(\mathbf{x})) = \frac{\partial f}{\partial \mathbf{x}} + \frac{\partial g}{\partial \mathbf{x}}$$

Product rule: $\frac{\partial}{\partial \mathbf{x}} (f(\mathbf{x}) g(\mathbf{x})) = \frac{\partial f}{\partial \mathbf{x}} g(\mathbf{x}) + f(\mathbf{x}) \frac{\partial g}{\partial \mathbf{x}}$

Chain rule: $\frac{\partial}{\partial \mathbf{x}} g(f(\mathbf{x})) = \frac{\partial g}{\partial f} \frac{\partial f}{\partial \mathbf{x}}$

With vectors and matrices, AB and BA are different things (see the [matrix multiplication as composition Note](#)). So we keep the order written above and check that the shapes fit.

A building block we need: the gradient of a dot product with a fixed vector $\mathbf{a}$. Since $\mathbf{a}^\top \mathbf{x} = a_1 x_1 + a_2 x_2$, the partial derivatives are a_1 and a_2 :

$$\frac{\partial}{\partial \mathbf{x}} \mathbf{a}^\top \mathbf{x} = \mathbf{a}^\top$$

1. **In words (product rule):** differentiate one factor at a time, keep the other, add.
2. **Formula:** for $f(\mathbf{x}) = (\mathbf{a}^\top \mathbf{x})(\mathbf{b}^\top \mathbf{x})$,

$$\nabla f = (\mathbf{b}^\top \mathbf{x}) \mathbf{a}^\top + (\mathbf{a}^\top \mathbf{x}) \mathbf{b}^\top$$

3. **Example:** $\mathbf{a} = [1, 2]$, $\mathbf{b} = [3, -1]$, $\mathbf{x} = [1, 1]$. Then $\mathbf{a}^\top \mathbf{x} = 3$ and $\mathbf{b}^\top \mathbf{x} = 2$:

$$\nabla f = 2[1, 2] + 3[3, -1] = [11, 1]$$

Check by multiplying out: $f = 3x_1^2 + 5x_1x_2 - 2x_2^2$ has partial derivatives $6x_1 + 5x_2 = 11$ and $5x_1 - 4x_2 = 1$.

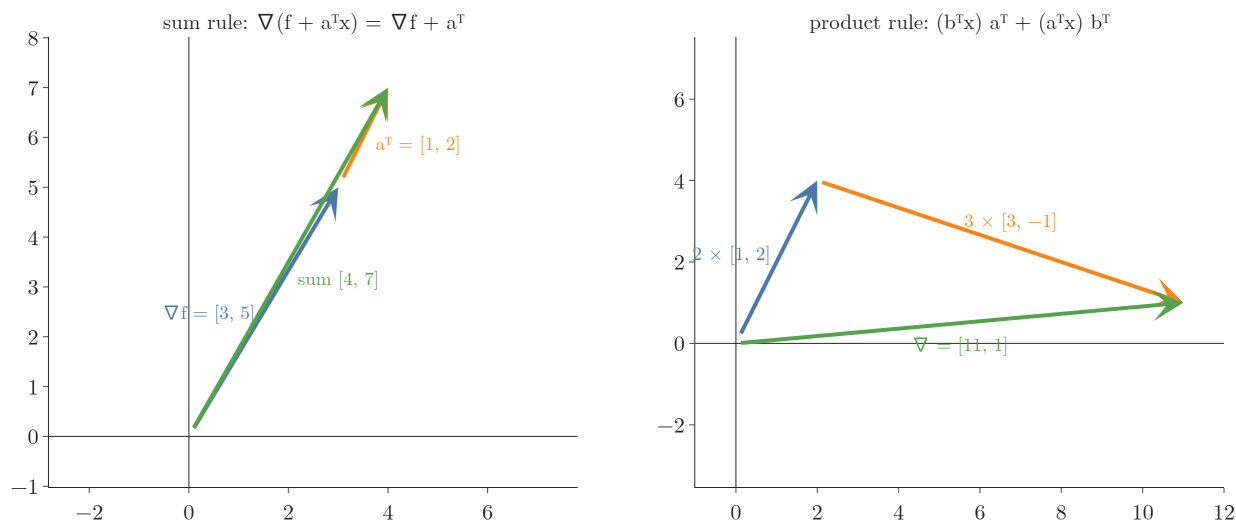


Figure 8: Gradient rules as additions of arrows at $\mathbf{x} = (1, 1)$. Left, sum rule: the gradient of $f + \mathbf{a}^\top \mathbf{x}$ is $\nabla f = [3, 5]$ plus $\mathbf{a}^\top = [1, 2]$, giving $[4, 7]$. Right, product rule: $2 \times [1, 2]$ plus $3 \times [3, -1]$ gives the gradient $[11, 1]$.

Figure 8 draws both rules as arrows added tip to tail. A gradient is a vector, so the sum rule adds two arrows. The product rule adds two arrows too, each one gradient scaled by the value of the other factor: $\mathbf{a}^\top$ scaled by 2 and $\mathbf{b}^\top$ scaled by 3.

6. The chain rule with several variables

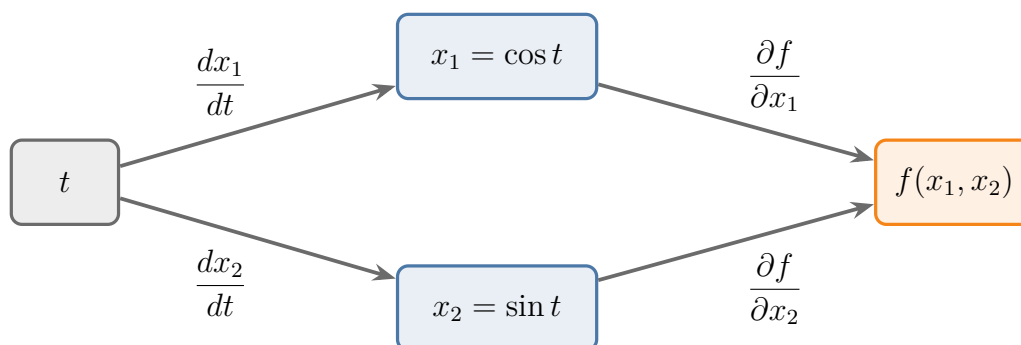
Key point: When an input affects f through several intermediate variables, we multiply along each path and add up the paths; written with gradients, this is a matrix product.

This rule is the **multivariate chain rule** (G-1281), the several-variable form of the **chain rule** (G-371).

6.1 One outer input, two paths

Key point: The rate of change of f with t is the sum over both paths of (slope of f along the path) times (rate of the intermediate variable): a row vector times a column vector.

Suppose x_1 and x_2 are themselves functions of one variable t , for example a point moving around a circle: $x_1 = \cos t$, $x_2 = \sin t$. How fast does $f(x_1, x_2)$ change as t changes? A change in t reaches f along two paths (Figure 9).



$$\frac{df}{dt} = \underbrace{\frac{\partial f}{\partial x_1} \frac{dx_1}{dt}}_{\text{path through } x_1} + \underbrace{\frac{\partial f}{\partial x_2} \frac{dx_2}{dt}}_{\text{path through } x_2}$$

Figure 9: The multivariate chain rule: multiply the derivatives along each path from t to f , then add the paths

1. **In words:** for each intermediate variable, multiply “how f responds to it” by “how it responds to t ”; add the results.

2. **Formula:**

$$\frac{df}{dt} = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} \end{bmatrix} \begin{bmatrix} \frac{dx_1}{dt} \\ \frac{dx_2}{dt} \end{bmatrix} = \frac{\partial f}{\partial x_1} \frac{dx_1}{dt} + \frac{\partial f}{\partial x_2} \frac{dx_2}{dt}$$

3. **Example:** our bowl on the circle. At $t = \pi/2$ the point is $(0, 1)$, where $\nabla f = [0 + 1, 0 + 4] = [1, 4]$. The point moves with velocity $[-\sin t, \cos t] = [-1, 0]$:

$$\frac{df}{dt} = 1 \cdot (-1) + 4 \cdot 0 = -1$$

At $t = 0$ the point is $(1, 0)$, $\nabla f = [2, 1]$, velocity $[0, 1]$, so $df/dt = 1$.

We can check the rule the long way, without it. Put the circle into the bowl first: $f = \cos^2 t + \cos t \sin t + 2 \sin^2 t$, a function of t alone. Using $\cos^2 t + \sin^2 t = 1$ and $\cos t \sin t = \frac{1}{2} \sin 2t$, it becomes $f = 1 + \sin^2 t + \frac{1}{2} \sin 2t$. The one-variable rules give

$$\frac{df}{dt} = 2 \sin t \cos t + \cos 2t = \sin 2t + \cos 2t$$

At $t = \pi/2$ the value is $0 + (-1) = -1$, and at $t = 0$ it is $0 + 1 = 1$: the same two answers. The long way needs a new round of algebra for every function. The chain rule gives the same result from two small pieces that are already known, the gradient and the velocity.

The first form is a 1×2 row times a 2×1 column, giving a 1×1 number. The chain rule is where the row-vector convention pays off: the gradient sits on the left and the shapes fit with no transposing.

6.2 Two outer inputs: a matrix of partial derivatives

Key point: With inputs s and t , the inner derivatives fill a 2×2 matrix, and the chain rule becomes (row gradient) times (matrix).

Now let x_1 and x_2 depend on two variables s and t . Each of $\partial f / \partial s$ and $\partial f / \partial t$ follows the path rule:

$$\frac{\partial f}{\partial s} = \frac{\partial f}{\partial x_1} \frac{\partial x_1}{\partial s} + \frac{\partial f}{\partial x_2} \frac{\partial x_2}{\partial s}, \quad \frac{\partial f}{\partial t} = \frac{\partial f}{\partial x_1} \frac{\partial x_1}{\partial t} + \frac{\partial f}{\partial x_2} \frac{\partial x_2}{\partial t}$$

1. **In words:** both equations at once are one row vector times one matrix.

2. **Formula:**

$$\frac{df}{d(s, t)} = \underbrace{\begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} \end{bmatrix}}_{\partial f / \partial \mathbf{x}} \underbrace{\begin{bmatrix} \frac{\partial x_1}{\partial s} & \frac{\partial x_1}{\partial t} \\ \frac{\partial x_2}{\partial s} & \frac{\partial x_2}{\partial t} \end{bmatrix}}_{\partial \mathbf{x} / \partial (s, t)}$$

3. **Example:** our bowl with $x_1 = s + t$ and $x_2 = st$, at $s = 1, t = 2$. Then $x_1 = 3, x_2 = 2$, so $\nabla f = [2 \cdot 3 + 2, 3 + 4 \cdot 2] = [8, 11]$. The inner derivatives are $\partial x_1 / \partial s = 1, \partial x_1 / \partial t = 1, \partial x_2 / \partial s = t = 2, \partial x_2 / \partial t = s = 1$:

$$\begin{bmatrix} 8 & 11 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 8 + 22 & 8 + 11 \end{bmatrix} = \begin{bmatrix} 30 & 19 \end{bmatrix}$$

The matrix in the middle is the first example of a **Jacobian** (G-980), the subject of the [Jacobian Note](#). The chain rule as a product of matrices mirrors the [matrix multiplication as composition Note](#): doing one function after another multiplies their matrices.

The notation also reads like fractions: $\partial \mathbf{x}$ appears “below” in the first factor and “above” in the second, and “cancels” to leave $\partial f / \partial (s, t)$. The fraction reading is a memory aid only; partial derivatives are not really fractions.

7. Checking a gradient numerically

Key point: Nudge each input by a small h and compare the finite-difference slope with the formula; a tiny relative error means the formula and code are probably right.

Hand-derived gradients are easy to get wrong, and a wrong gradient makes training fail quietly. The definition of the partial derivative gives a simple test, called **gradient checking** (G-860):

1. **In words:** estimate each partial derivative with a small step h (say 10^{-4}), then compare the whole estimated gradient with the formula’s.
2. **Formula:** with d_i^h the finite-difference estimate and d_i the formula’s value for input i ,

$$\text{relative error} = \sqrt{\frac{\sum_i (d_i^h - d_i)^2}{\sum_i (d_i^h + d_i)^2}}$$

A value below about 10^{-6} means the formula is very probably correct (MML §5.2).

3. **Example:** the squared-error loss $L(m, b) = \sum (y_i - mx_i - b)^2$ of the [gradient descent Note](#), on three points $x = [1, 2, 3]$, $y = [2, 4, 5]$, at $m = 1, b = 0$. The residuals are $[1, 2, 2]$, so the formulas give

$$\frac{\partial L}{\partial m} = -2 \sum r_i x_i = -2(1 + 4 + 6) = -22, \quad \frac{\partial L}{\partial b} = -2 \sum r_i = -10$$

Central differences with $h = 10^{-4}$ give $[-22.0000, -10.0000]$, a relative error of about 10^{-13} .

Python:

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])
y = np.array([2.0, 4.0, 5.0])
```

```

def loss(p):
    m, b = p
    return np.sum((y - m * x - b) ** 2)

def grad(p):
    r = y - p[0] * x - p[1]
    return np.array([-2 * np.sum(r * x), -2 * np.sum(r)])

p, h = np.array([1.0, 0.0]), 1e-4
num = np.array([(loss(p + h * e) - loss(p - h * e)) / (2 * h)
                for e in np.eye(2)])
rel = np.linalg.norm(num - grad(p)) / np.linalg.norm(num + grad(p))
num, grad(p), rel      # [-22, -10], [-22, -10], about 2.5e-13
np.eye(2) gives the rows [1,0] and [0,1], so p + h * e nudges one parameter at a time.

```

Figure 10 shows how the size of the step h affects the check on this loss. The one-sided difference $(L(\mathbf{p} + h\mathbf{e}) - L(\mathbf{p}))/h$ is off by an amount that shrinks in proportion to h ; it only passes the 10^{-6} line for h below about 3×10^{-6} . The central difference has no such error on this loss: the loss is quadratic, and for a quadratic the central difference is exact. What is left is rounding error, which grows as h gets tiny, because two almost equal losses are subtracted (the Extra on central differences in the [derivatives of one variable Note](#)).

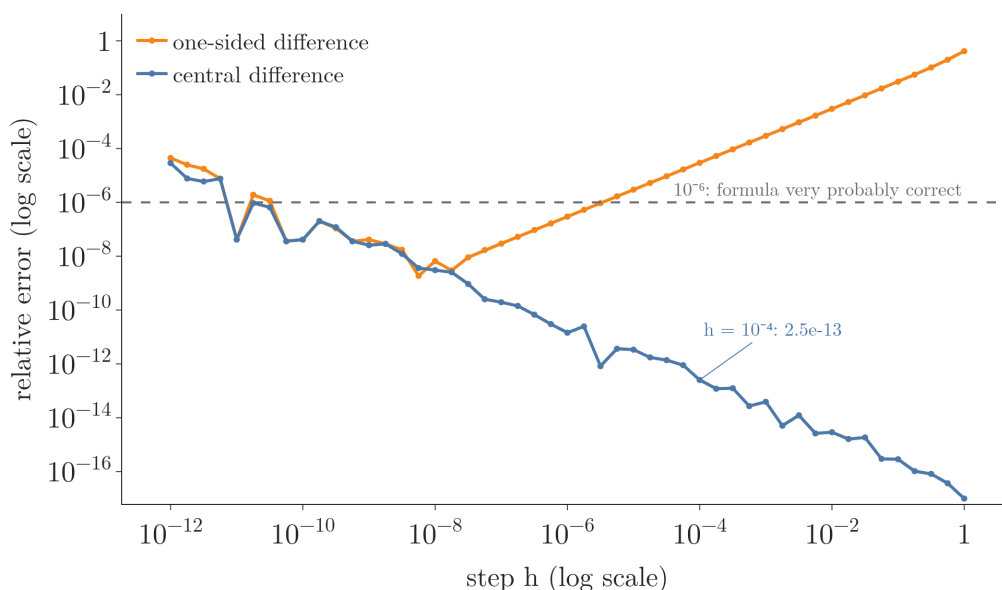


Figure 10: Relative error of the numerical gradient against the step h , on the three-point loss at $m = 1$, $b = 0$. Orange: one-sided difference, error shrinking with h until rounding takes over. Blue: central difference, exact for this quadratic loss except for rounding, which grows as h shrinks. Dashed: the 10^{-6} threshold.

Deep learning libraries compute gradients automatically, and they ship the same test for checking them: PyTorch’s `torch.autograd.gradcheck` compares the automatic gradients with finite differences (PyTorch docs, “Gradcheck mechanics”).

8. Summary

Object	Shape	Example for $f = x_1^2 + x_1x_2 + 2x_2^2$ at $(1, 1)$
Function value $f(\mathbf{x})$	number	4
Partial derivative $\partial f / \partial x_i$	number	3 and 5
Gradient ∇f	$1 \times n$ row	$[3, 5]$
Chain rule $\partial f / \partial (s, t)$	$(1 \times 2)(2 \times 2) = 1 \times 2$	$[30, 19]$ at $s = 1, t = 2$

- A partial derivative moves one input and holds the others fixed; it is the slope of one slice of the surface.
- The gradient collects all partial derivatives; it points straight uphill, at right angles to the contour lines, and gradient descent steps against it.
- Sum, product and chain rules carry over to vectors, with the order of factors kept.
- The multivariate chain rule adds the products along every path; with gradients as rows, it is matrix multiplication.
- Gradient checking compares a formula with finite differences.

9. Sources

Built from

- Deisenroth, M. P., Faisal, A. A. and Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. Section 5.2, remark on verifying a gradient implementation (MML).
- Khan Academy, “Partial derivatives, introduction”, YouTube, <https://www.youtube.com/watch?v=AX...> (Figure 3)
- Khan Academy, “Partial derivatives and graphs”, YouTube, <https://www.youtube.com/watch?v=dfvnC...>
- Khan Academy, “Gradient”, YouTube, <https://www.youtube.com/watch?v=tIpKfDc295M>
- Khan Academy, “Gradient and graphs”, YouTube, https://www.youtube.com/watch?v=_-02ze7tf08 (Figure 5)
- Khan Academy, “Gradient and contour maps”, YouTube, <https://www.youtube.com/watch?v=ZTbTY...> (Figure 6)
- Khan Academy, “Directional derivative”, YouTube, https://www.youtube.com/watch?v=N_ZRcLheNv...
- Khan Academy, “Why the gradient is the direction of steepest ascent”, YouTube, <https://www.youtube.com/watch?v=TEB2z7ZIRAw> (Figure 7 recreates its idea with our own function and code)
- Khan Academy, “Multivariable chain rule”, YouTube, <https://www.youtube.com/watch?v=NO3AqAaA...>

Other references

- PyTorch documentation. “Gradcheck mechanics” and `torch.autograd.gradcheck`.

10. Key terms

Term	Meaning
Function of several variables	$f : \mathbb{R}^n \rightarrow \mathbb{R}$: a vector of n numbers in, one number out
Gradient as a row vector	$\nabla f = [\partial f / \partial x_1, \dots, \partial f / \partial x_n] \in \mathbb{R}^{1 \times n}$, the convention that makes the chain rule a matrix product
Nabla (∇)	The symbol for the gradient
Directional derivative	The slope of f along a unit vector $\mathbf{u}$: $\nabla f \cdot \mathbf{u}$
Steepest ascent	The direction in which f increases fastest: the direction of the gradient
Multivariate chain rule	The derivative through intermediate variables: multiply along each path and add the paths; a row gradient times a matrix of inner derivatives
Gradient checking	Testing a gradient formula against finite-difference estimates, using the relative error